Self-Organizing Maps (SOM) Tutorial with the Iris Dataset

Name: Priston Dlima Student_Id:24166078

1. Introduction

Self-Organizing Maps (SOMs) is an unsupervised neural network algorithm that was presented by **Teuvo Kohonen** in 1982. They provide a way to visualize and explore high-dimensioned data by drawing it to a two-dimensional grid of neurons while retaining the topological relationships among the data points; In contrast to the traditional clustering approaches, SOMs also have spatial structure that reveals the clusters and patterns for better interpretation.

In this tutorial, we focus on applying SOMs to the **Iris dataset**, one of the classic data sets that contains 150 5th-grade elementary school children; samples of iris flowers. Each sample contains four numerical properties or features namely sepal length, sepal width, petal length and petal width. The flowers are classified in three species: Iris-setosa, Iris-versicolor and Iris-virginica.

2. Objectives

The goals of this tutorial are to:

- Explain the concept and mechanism of Self-Organizing Maps.
- Demonstrate SOM training and visualization using Python.
- Provide insight into feature relationships using component planes.
- Highlight practical applications of SOMs in clustering and exploratory data analysis.
- Show how to interpret SOM results for real-world datasets.

3. SOM Concepts

A SOM is a 2D grid of neurons where each neuron is associated with a weight vector equal to the number of dimensions of the input data. When all presented data point:

- The neuron with the closest weight vector to the input, called the **Best Matching Unit (BMU)**, is identified.
- The BMU and its neighbors change their values so that they move closer to the input.
- This process repeats itself with multiple iterations resulting in a topologically organized map such that similar samples illuminate similar regions of the map.

The key SOM parameters include:

- **Grid size:** Determines the number of neurons in the 2D map.
- **Learning rate:** Controls how much weights are updated each iteration.

- **Neighborhood radius (sigma):** Defines the influence of the BMU on neighboring neurons.
- **Iterations:** Number of training steps, affecting convergence quality.

4. Dataset Preparation

It is important to normalize the data in order to train a SOM because some features may have different scales. Standardization assures that each feature would be the same in making distance calculated to find the BMU.

```
from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler


# Load the Iris dataset

data = load_iris()

X = data.data


# Standardize the features

X = StandardScaler().fit_transform(X)

y = data.target

features = ['Sepal length', 'Sepal width', 'Petal length', 'Petal width']
```

5. SOM Training Implementation

Instead of relying on outside libraries such as MiniSom, we create a simple SOM from scratch so as to have full control and transparency. Weights are initialized randomly and a Gaussian neighborhood function is used to update weights of BMU and its neighbors:

```
import numpy as np


# SOM parameters

grid_size = 7

input_len = X.shape[1]

iterations = 1000

learning_rate = 0.5

sigma = 1.0
```

```
# Initialize random weights
weights = np.random.rand(grid_size, grid_size, input_len)

# Gaussian neighborhood function
def neighborhood(dist_sq, sigma):
    return np.exp(-dist_sq / (2 * sigma**2))

# Training loop
for it in range(iterations):
    sample = X[np.random.randint(0, X.shape[0])]
    # Find BMU
    diff = weights - sample
    dist_sq = np.sum(diff**2, axis=2)
    bmu_idx = np.unravel_index(np.argmin(dist_sq), (grid_size, grid_size))
    # Update weights
    for i in range(grid_size):
        for j in range(grid_size):
            dist_to_bmu_sq = (i - bmu_idx[0])**2 + (j - bmu_idx[1])**2
            h = neighborhood(dist_to_bmu_sq, sigma)
            weights[i,j,:] += learning_rate * h * (sample - weights[i,j,:])
```

This process enables the SOM to self-organize to activate similar data points among neurons that are close to each other. After training, the map can be used for visualization and analysis.

6. Visualizing the SOM

6.1 U-Matrix

The Unified Distance Matrix (U-Matrix) is the average of distances between each of its immediate neighbours. Large distances represent boundaries of clusters, but smaller distances represent homogeneity within clusters.

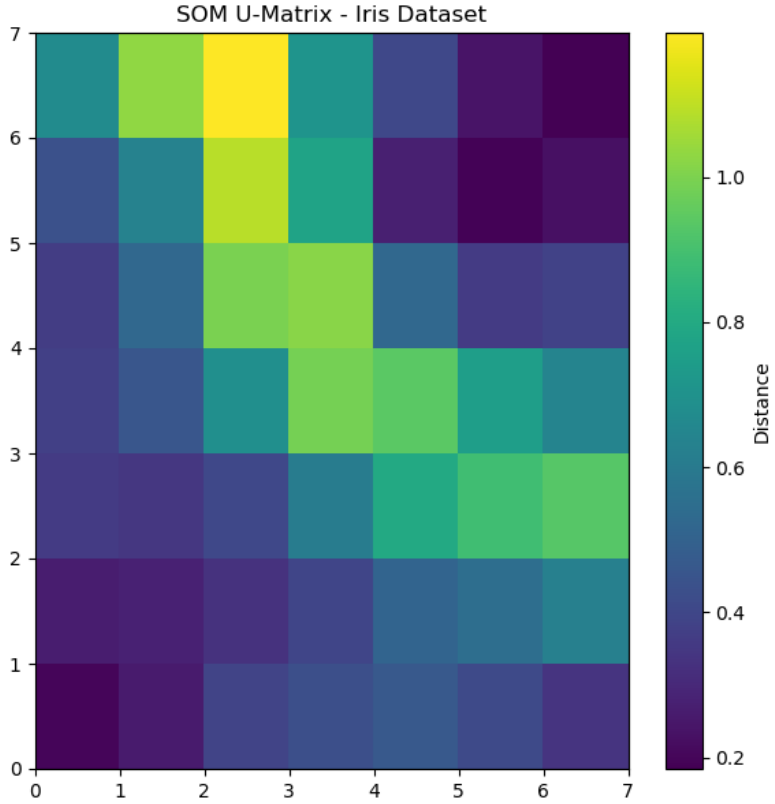

Figure 1: SOM U-Matrix of the Iris dataset. Darker areas highlight potential cluster boundaries.

6.2 Label Overlay

Mapping the actual class labels on top of the SOM a visual evaluation of how well the SOM separates different classes can be performed. If neurons representing the same species are grouped together, it is evidence of successful clustering.

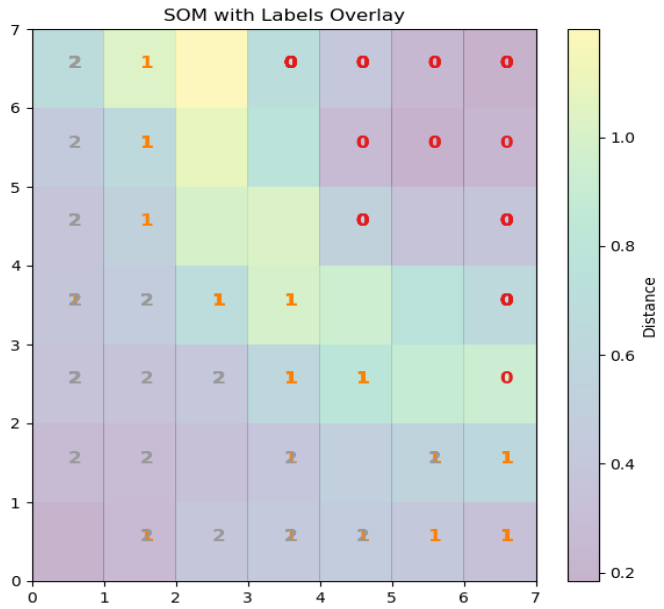

Figure 2: Class labels overlayed on the SOM. Each number corresponds to a species (0: Setosa, 1: Versicolor, 2: Virginica).

6.3 Component Planes

Component planes show the distribution of single features at the SOM. They facilitate the understanding of what features are helping in the clustering the most.

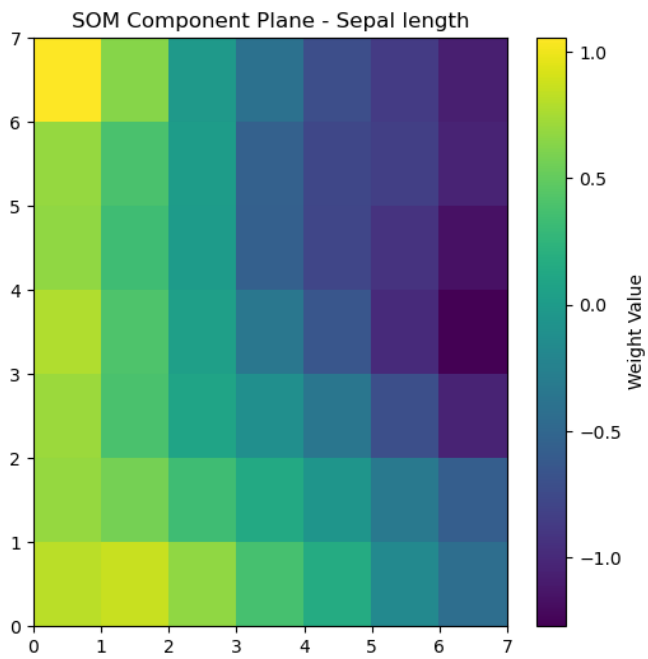

Figure 3a: Sepal Length contribution to SOM clustering

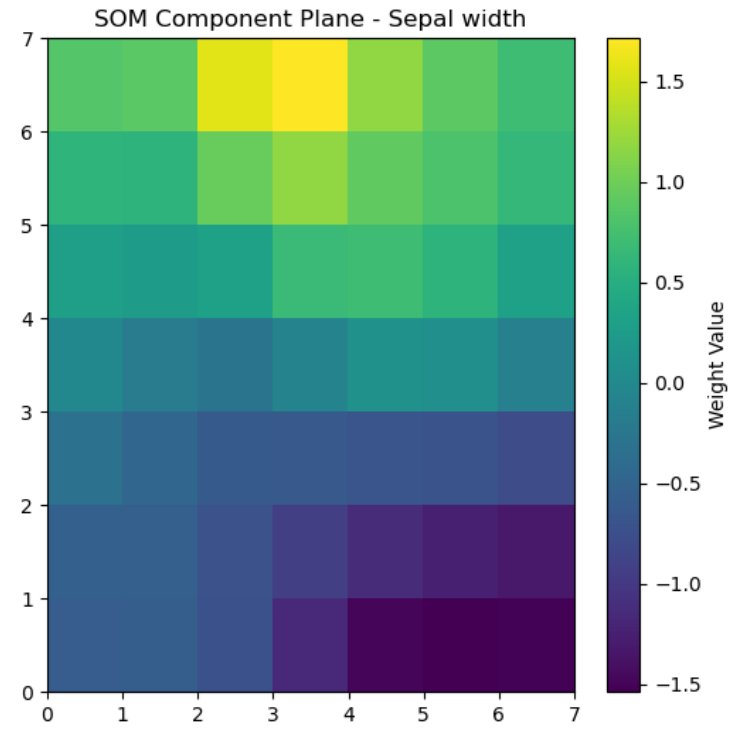

Figure 3b: Sepal Width contribution

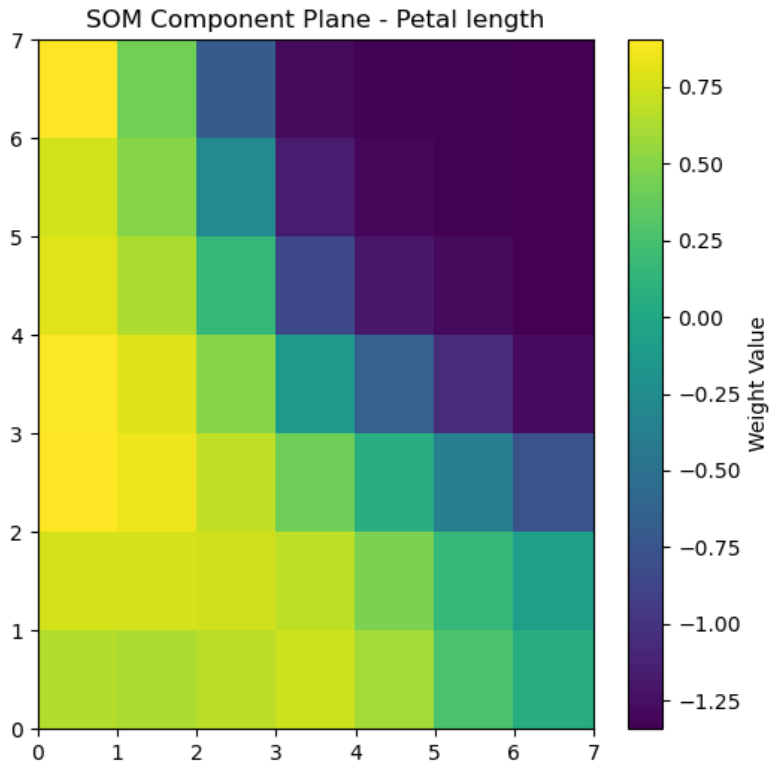

Figure 3c: Petal Length contribution

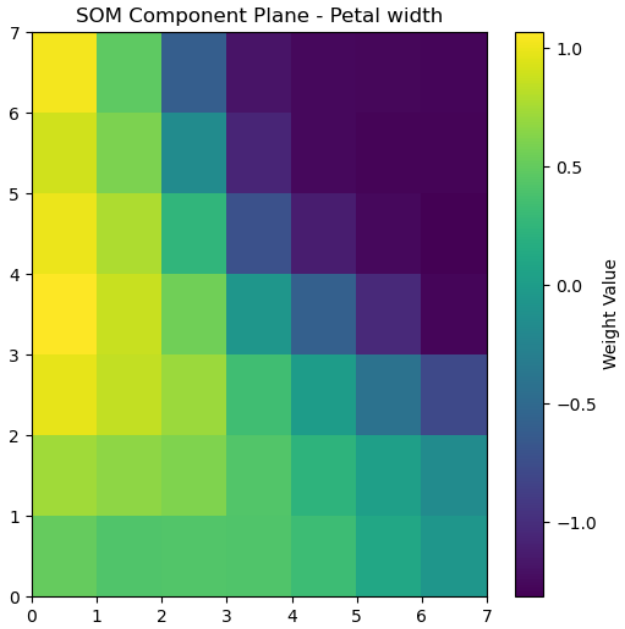

Figure 3d: Petal Width contribution

6.4 Cluster Assignment Map

Each neuron is assigned the most common class in the samples that it represents. This visualization gives a summary of the dominant species for each neuron and gives a fairly good overview of the quality of the clustering.

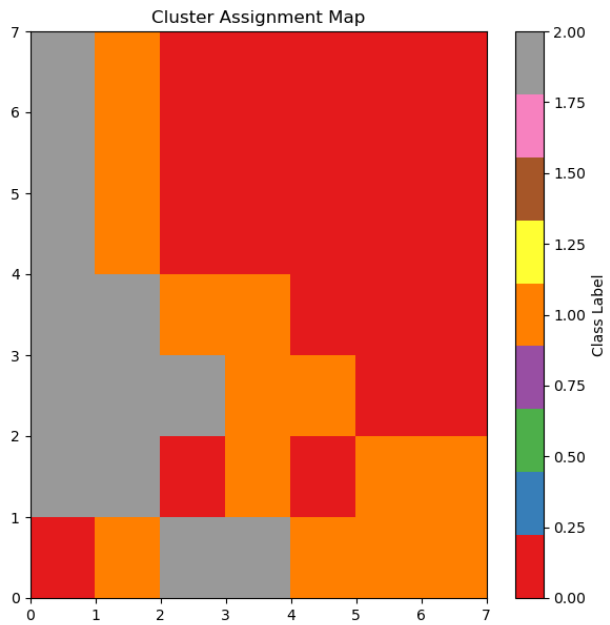

Figure 4: Cluster assignment map. Each neuron is colored by its dominant class.

7. Insights and Analysis

- SOM efficiently reduces high-dimensional data into a 2D map while preserving relationships.
- U-Matrix highlights cluster boundaries, revealing hidden structure in the data.
- Label overlay verifies class separation and cluster purity.
- Component planes indicate the influence of each feature on the self-organization process.
- Cluster assignment maps help in summarizing the distribution of classes across the SOM grid.

8. Accessibility and Design Choices

- Used colorblind-friendly colormaps (viridis) to make figures interpretable for all users.
- Alt text added to all images for screen readers.
- Clear font selection and figure captions improve readability.
- Interactive and descriptive tutorials help learners understand each step of SOM implementation.

9. Code Availability and GitHub Repository

The full code for training the SOM, generating figures, and creating cluster maps is available on GitHub: https://github.com/Priston11/SOM-Tutorial-Iris-Dataset/tree/main. The repository includes:

- Jupyter Notebook with complete code and explanations.
- Figures used in this tutorial.
- README file and license.
- Instructions for running the tutorial locally.

10. References

- Kohonen, T. (1990). *The Self-Organizing Map*. Proceedings of the IEEE, 78(9), 1464-1480.
- Scikit-learn Iris dataset: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html
- MiniSom Documentation: https://github.com/JustGlowing/minisom
- Python Numpy Documentation: https://numpy.org/doc/stable/